

nonetype

CSCI
134

expressions
so far

type	example	value
int	7	7
float	3.14	3.14
str	"williams"	"williams"
bool	3 > 4	False
function	abs	<function abs(x)>

every expression has
a value and a type

interactive python tells us the value
of any expression we type using `Out[]`:

```
In [1]: pow(2, 3)
Out[1]: 8

In [2]: 4 / 3
Out[2]: 1.3333333333333333

In [3]: "hi"
Out[3]: 'hi'

In [4]: lambda x: 2*x
Out[4]: <function __main__.<lambda>(x)>

In [5]: print("hello")
hello
```

interactive python tells us the value
of any expression we type using `Out[]`:

where's
`Out[5]`? →

```
In [1]: pow(2, 3)
Out[1]: 8

In [2]: 4 / 3
Out[2]: 1.3333333333333333

In [3]: "hi"
Out[3]: 'hi'

In [4]: lambda x: 2*x
Out[4]: <function __main__.<lambda>(x)>

In [5]: print("hello")
hello
```

interactive python tells us the value
of any expression we type using `Out[]`:

does this mean

```
print("hello")
```

has no value?



```
In [1]: pow(2, 3)
Out[1]: 8

In [2]: 4 / 3
Out[2]: 1.3333333333333333

In [3]: "hi"
Out[3]: 'hi'

In [4]: lambda x: 2*x
Out[4]: <function __main__.<lambda>(x)>

In [5]: print("hello")
hello
```

yes and no...

```
In [1]: pow(2, 3)
```

```
Out[1]: 8
```

```
In [2]: 4 / 3
```

```
Out[2]: 1.3333333333333333
```

```
In [3]: "hi"
```

```
Out[3]: 'hi'
```

```
In [4]: lambda x: 2*x
```

```
Out[4]: <function __main__.<lambda>(x)>
```

```
In [5]: print("hello")
```

```
hello
```

```
In [1]: print(pow(2, 3))
```

```
8
```

```
In [2]: print(4 / 3)
```

```
1.3333333333333333
```

```
In [3]: print("hi")
```

```
hi
```

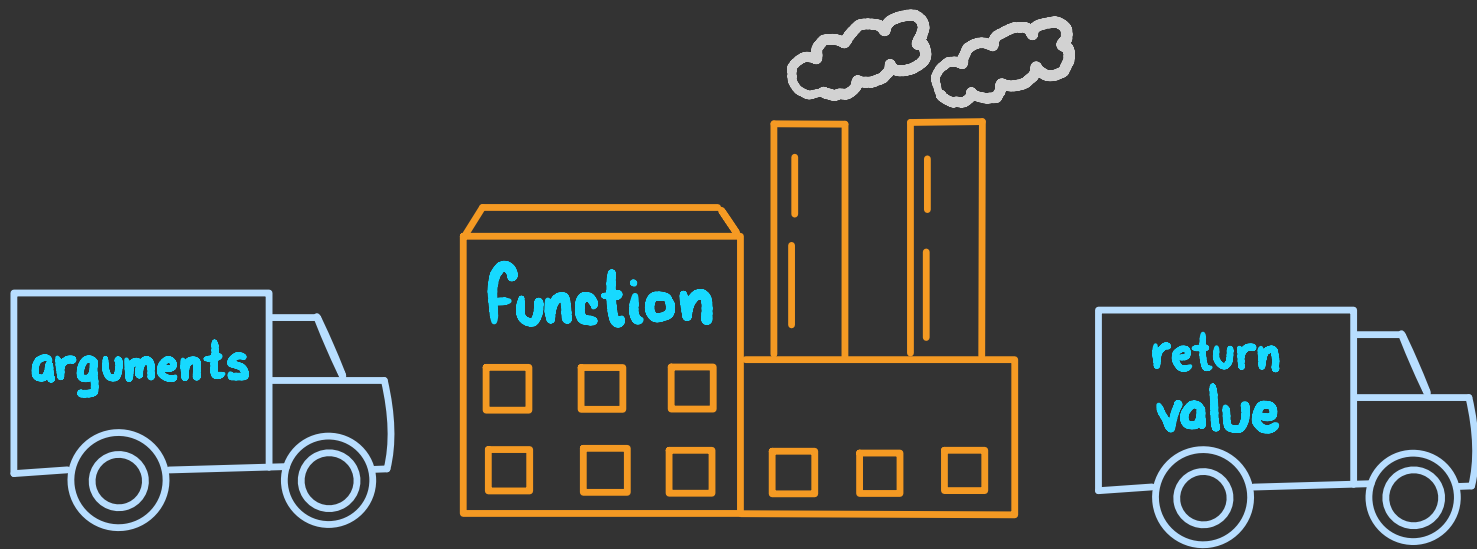
```
In [4]: print(lambda x: 2*x)
```

```
<function <lambda> at 0x7f836964eb00>
```

```
In [5]: print(print("hello"))
```

```
hello
```

```
None
```



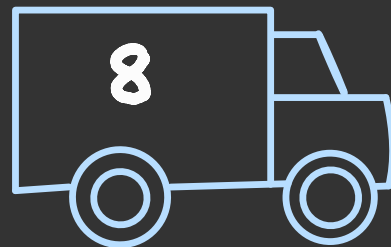
```
In [1]: pow(2, 3)  
Out[1]: 8
```



arguments



function

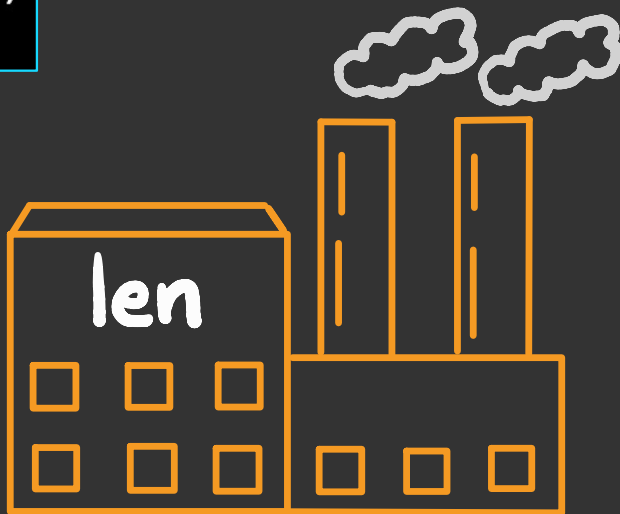


return value

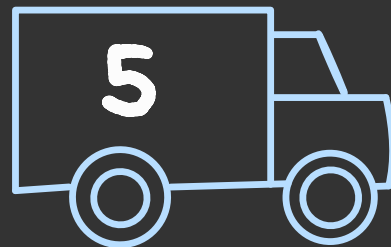
```
In [1]: len("hello")  
Out[1]: 5
```



arguments



function

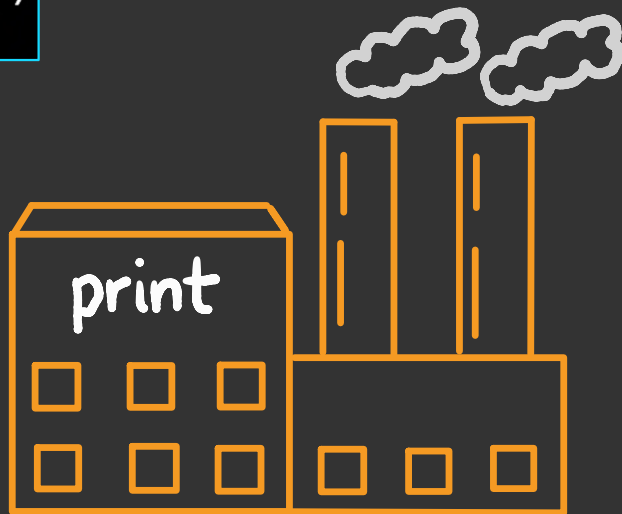


return value

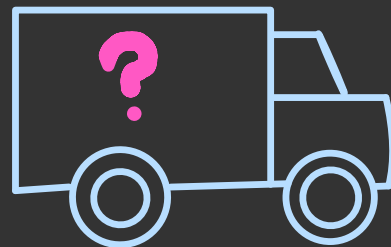
```
In [5]: print("hello")  
hello
```



arguments



function

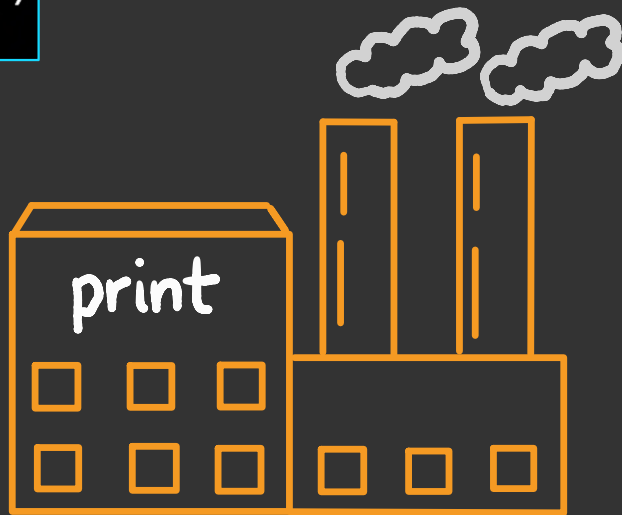


return value

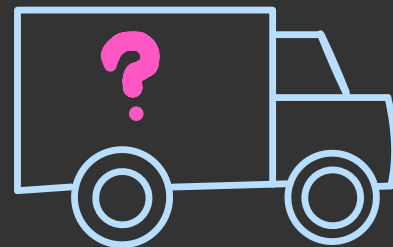
```
In [5]: print("hello")  
hello
```



arguments



function



return value

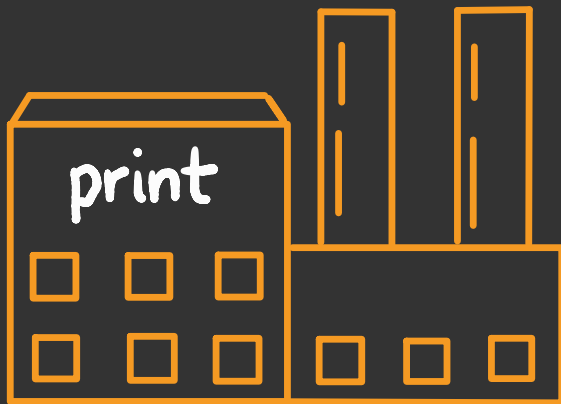
is this truck
empty?



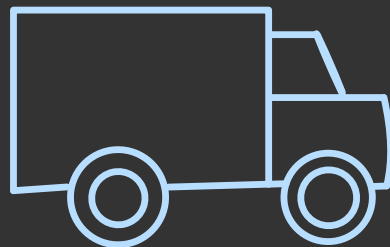
not exactly



arguments

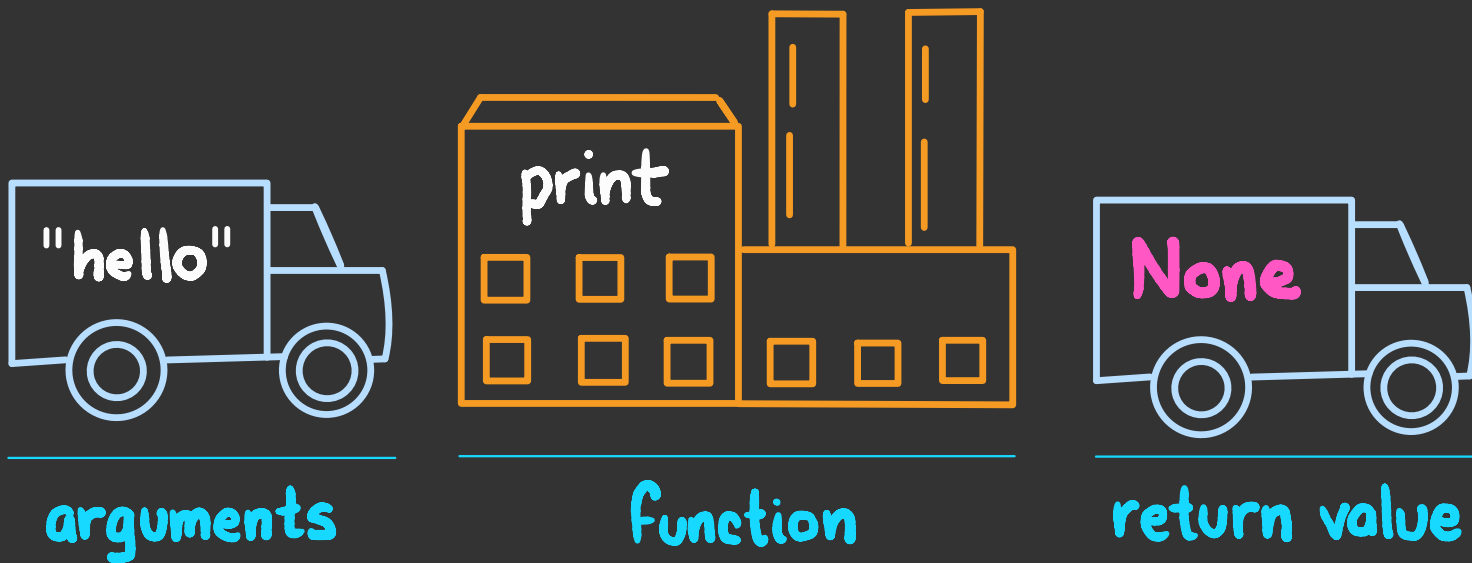


function



return value

its return value is a special expression called **None**





```
In [1]: x = print("hello")  
hello
```

```
In [2]: print(x)  
None
```

what you might
expect

```
In [1]: "hello"  
Out[1]: 'hello'
```

```
In [2]: 3.8  
Out[2]: 3.8
```

```
In [3]: None  
Out[3]: None
```

```
In [4]: True  
Out[4]: True
```

what actually
happens

```
In [1]: "hello"  
Out[1]: 'hello'
```

```
In [2]: 3.8  
Out[2]: 3.8
```

```
In [3]: None
```

```
In [4]: True  
Out[4]: True
```

interactive
python hides
the value of
an expression
if its value is

None

guess what happens

```
In [1]: def square(num):  
...:     pow(num, 2)  
...:
```

```
In [2]: square(5)
```

[redacted]

guess what happens

```
In [1]: def square(num):  
...:     pow(num, 2)  
...:
```

```
In [2]: square(5)
```

```
In [3]: █
```

if you don't **explicitly** specify the return value of a function, then python assumes it is **None**

```
In [1]: def square(num):  
...:     pow(num, 2)  
...:     return None
```

assumed

Out[2]: None

suppressed

```
In [2]: square(5)
```

```
In [3]: █
```

forgetting to specify the return value is a common error

```
In [1]: def square(num):  
...:     pow(num, 2)  
...:
```

```
In [2]: square(5)
```

```
In [3]: █
```

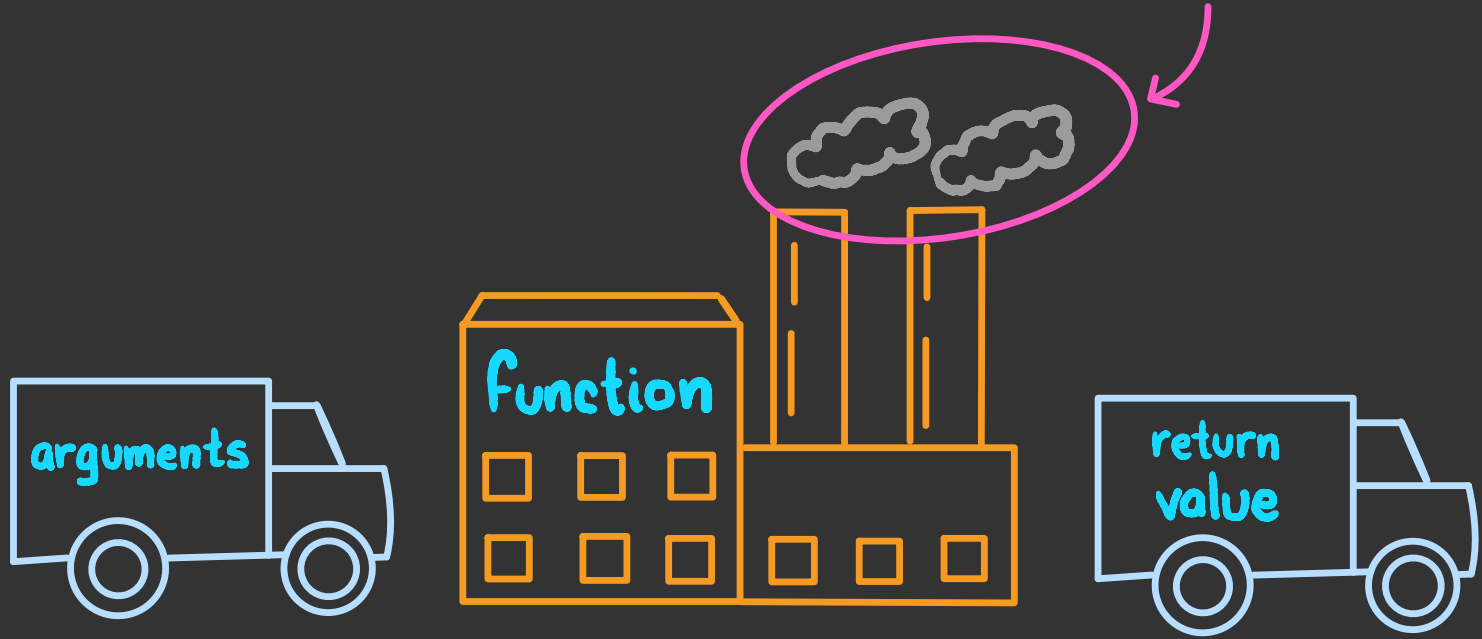
```
In [1]: def square(num):  
...:     return pow(num, 2)  
...:
```

```
In [2]: square(5)
```

```
Out[2]: 25
```

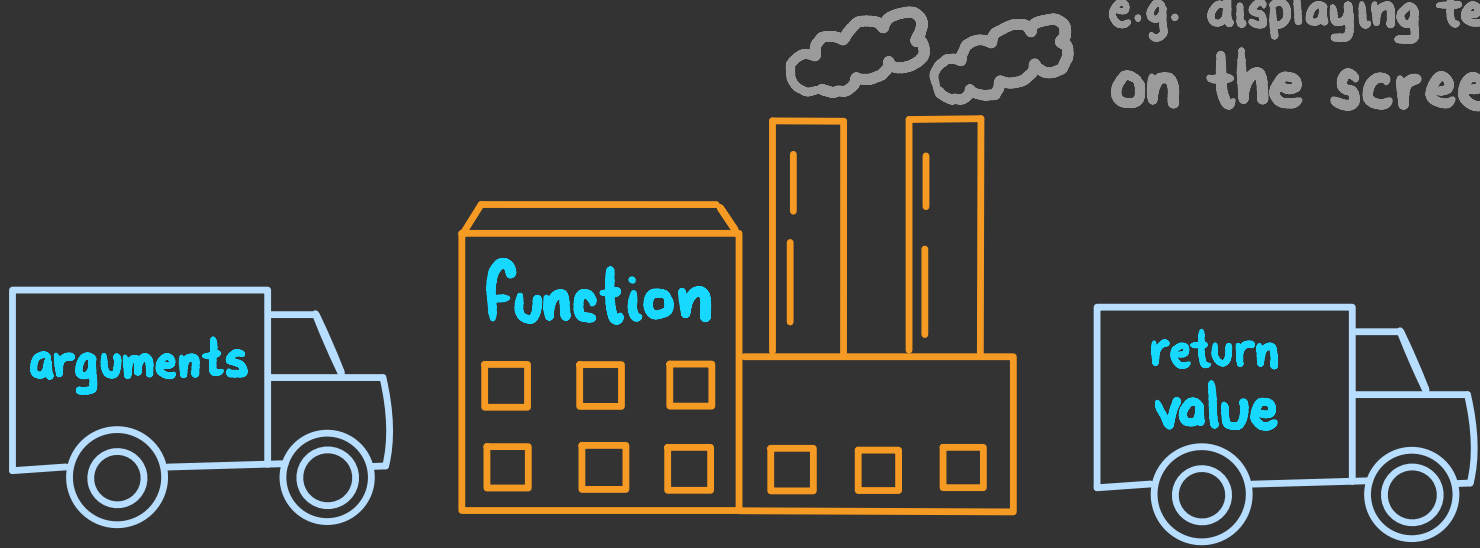
```
In [3]: █
```

Functions can also have side effects

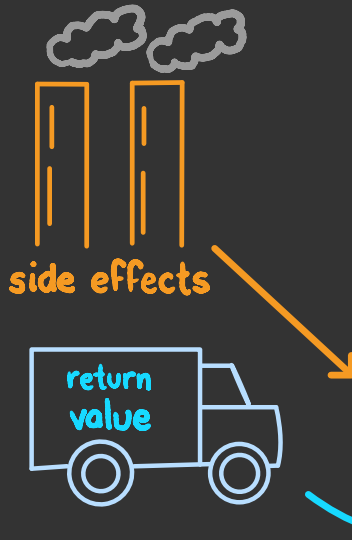


Functions can also have **side effects**

e.g. displaying text
on the screen



Functions can also have side effects



```
In [1]: def square(num):  
...:     print("let's square a number!")  
...:     result = pow(num, 2)  
...:     return result  
...:
```

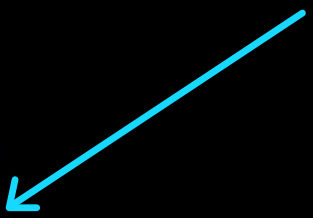
```
In [2]: square(5)  
let's square a number!  
Out[2]: 25
```

the value of **this expression** is 25,
so we can add one to it

```
In [1]: def square(num):  
...:     print("let's square a number!")  
...:     result = pow(num, 2)  
...:     return result  
...:
```

```
In [2]: square(5)  
let's square a number!  
Out[2]: 25
```

```
In [3]: 1 + square(5)  
let's square a number!  
Out[3]: 26
```

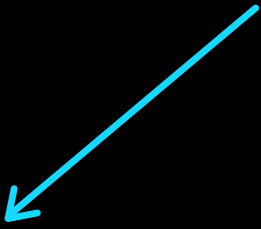


the side effect occurs
every time we call square

```
In [1]: def square(num):  
...:     print("let's square a number!")  
...:     result = pow(num, 2)  
...:     return result  
...:
```

```
In [2]: square(5)  
let's square a number!  
Out[2]: 25
```

```
In [3]: 1 + square(5)  
let's square a number!  
Out[3]: 26
```



guess what happens

```
In [1]: def square(num):  
...:     print("let's square a number!")  
...:     result = pow(num, 2)  
...:
```

```
In [2]: square(5)  
let's square a number!
```

```
In [3]: 1 + square(5)
```

[redacted]

guess what happens

```
In [1]: def square(num):  
...:     print("let's square a number!")  
...:     result = pow(num, 2)  
...:
```

```
In [2]: square(5)  
let's square a number!
```

```
In [3]: 1 + square(5)  
let's square a number!
```

```
TypeError                                Traceback (most recent call last)  
Cell In[3], line 1  
----> 1 1 + square(5)
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

the side effect still occurs

```
In [1]: def square(num):  
...:     print("let's square a number!")  
...:     result = pow(num, 2)  
...:
```

```
In [2]: square(5)  
let's square a number!
```

```
In [3]: 1 + square(5)  
let's square a number!
```



```
TypeError  
Cell In[3], line 1  
----> 1 1 + square(5)
```

Traceback (most recent call last)

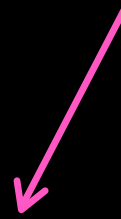
```
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

but we get an **error** when we
try to add 1 and None

```
In [1]: def square(num):  
...:     print("let's square a number!")  
...:     result = pow(num, 2)  
...:
```

```
In [2]: square(5)  
let's square a number!
```

```
In [3]: 1 + square(5)  
let's square a number!
```



```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[3], line 1  
----> 1 1 + square(5)  
  
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

but we get an **error** when we
try to add 1 and None

```
In [1]: 1 + None
```

```
-----  
TypeError  
Cell In[1], line 1  
----> 1 1 + None
```

Traceback (most recent call last)

```
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

so why did **this** happen?

```
In [1]: pow(2, 3)
```

```
Out[1]: 8
```

```
In [2]: 4 / 3
```

```
Out[2]: 1.3333333333333333
```

```
In [3]: "hi"
```

```
Out[3]: 'hi'
```

```
In [4]: lambda x: 2*x
```

```
Out[4]: <function __main__.<lambda>(x)>
```

```
In [5]: print("hello")
```

```
hello
```

```
In [1]: print(pow(2, 3))
```

```
8
```

```
In [2]: print(4 / 3)
```

```
1.3333333333333333
```

```
In [3]: print("hi")
```

```
hi
```

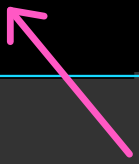
```
In [4]: print(lambda x: 2*x)
```

```
<function <lambda> at 0x7f836964eb00>
```

```
In [5]: print(print("hello"))
```

```
hello
```

```
None
```



let's evaluate the expression tree



Diagram illustrating the expression tree for the code snippet:

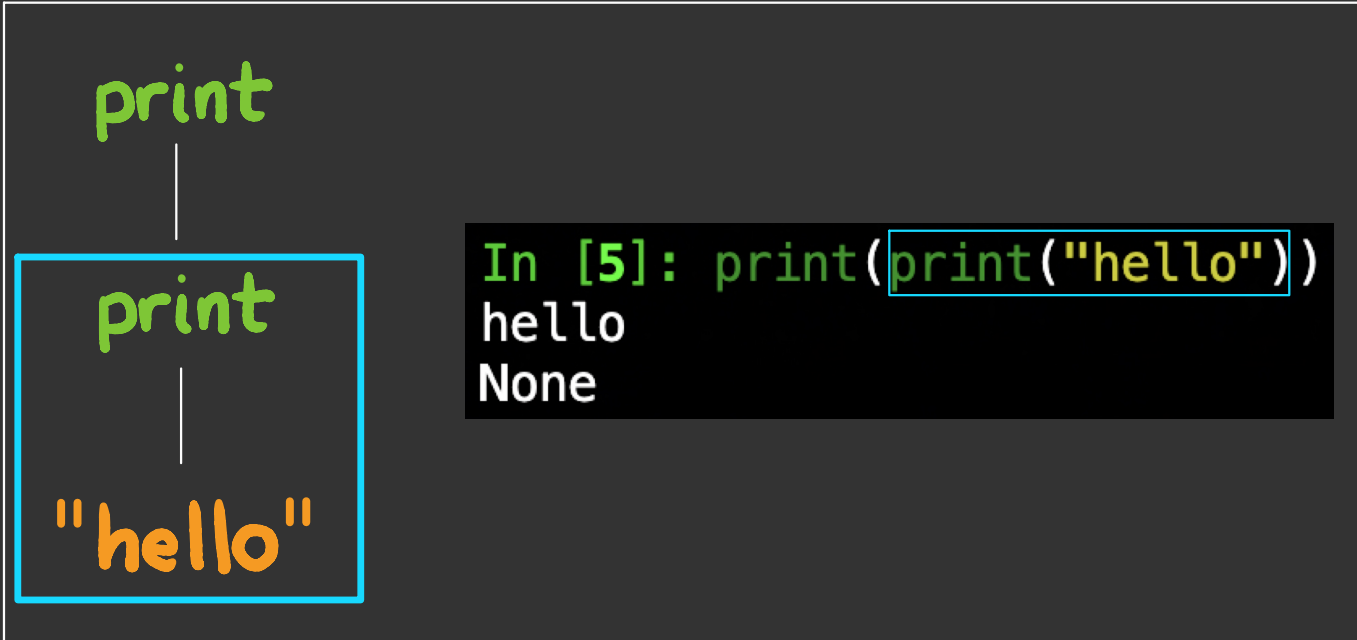
```
graph TD; A[print] --- B[print]; B --- C["hello"];
```

The tree structure is:

- print (green)
- |
- print (green)
- |
- "hello" (orange)

```
In [5]: print(print("hello"))  
hello  
None
```

First, python evaluates this expression



as a side effect, the **print** function displays its argument on the screen

print

print

"hello"

```
In [5]: print(print("hello"))
```

hello

None



the return value of the
print function is None

print

None

None

```
In [5]: print(print("hello"))  
hello  
None
```

next, python evaluates this expression

print

|

None

None

```
In [5]: print(print("hello"))  
hello  
None
```

as a side effect, the **print** function displays its argument on the screen

print

None

None

```
In [5]: print("hello")
```

hello

None



the return value of the
print function is None

None

None

```
In [5]: print(print("hello"))  
hello  
None  
Out[5]: None
```

but interactive python **suppresses**
the notification of None values

None

None

```
In [5]: print(print("hello"))  
hello  
None  
print(print("hello"))
```

so we end up with this

```
In [5]: print(print("hello"))  
hello  
None
```

return and print are often confused

return specifies the return value of a function

print displays a value on the screen

```
In [1]: def greeting(name):  
...:     return "hello " + name  
...:
```

```
In [2]: greeting("mark") + "!"  
Out[2]: 'hello mark!'
```

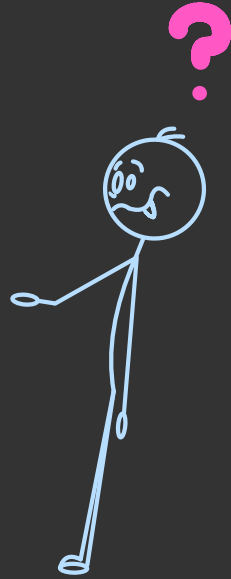
```
In [1]: def greeting(name):  
...:     print("hello " + name)  
...:
```

```
In [2]: greeting("mark") + "!"  
hello mark
```

```
TypeError  
Cell In[2], line 1  
----> 1 greeting("mark") + "!"
```

every expression has
a **value** and a **type**

what is None's type?



the type of None is `NoneType`

```
In [1]: pow(2, 3)
```

```
Out[1]: 8
```

```
In [2]: 4 / 3
```

```
Out[2]: 1.3333333333333333
```

```
In [3]: "hi"
```

```
Out[3]: 'hi'
```

```
In [4]: lambda x: 2*x
```

```
Out[4]: <function __main__.<lambda>(x)>
```

```
In [5]: print("hello")
```

```
hello
```

```
In [1]: type(pow(2, 3))
```

```
Out[1]: int
```

```
In [2]: type(4 / 3)
```

```
Out[2]: float
```

```
In [3]: type("hi")
```

```
Out[3]: str
```

```
In [4]: type(lambda x: 2*x)
```

```
Out[4]: function
```

```
In [5]: type(print("hello"))
```

```
hello
```

```
Out[5]: NoneType
```



expressions so far

type	example	value
int	7	7
float	3.14	3.14
str	"williams"	"williams"
bool	3 > 4	False
function	abs	<function abs(x)>
NoneType	print("hi")	None